

66. Cookies aur `document.cookie`

Cookies browser me seedhe store hone wale **chhote strings** hain. HTTP protocol ka hissa (RFC 6265).

Aam taur par server response me `Set-Cookie` header se set karta hai. Phir browser same domain ki (lagbhag) har request me use automatically `Cookie` header me bhejta hai.

Sabse common use: authentication

1. Sign-in par server `Set-Cookie` se unique "session identifier" wala cookie deta hai.
2. Agli request par browser cookie bhejta hai.
3. Server ko pata chal jaata hai kaun hai.

Browser me JS se bhi `document.cookie` ke zariye access kar sakte hain.

Padhna: `document.cookie`

```
js
alert(document.cookie); // cookie1=value1; cookie2=value2;...
```

`name=value` jodiyan, `;` se alag. Kisi ek ko dhundhne ke liye `;` se split karo (ya regex).

Likhna: `document.cookie = "..."`

Ye normal data property nahi, **accessor (getter/setter)** hai. Assignment special treat hota hai: **sirf wahi cookie update hota hai jiska naam likha hai, baaki nahi.**

```
js
document.cookie = "user=John"; // sirf 'user' update
alert(document.cookie); // saare cookies dikhte hain
```

Special characters (spaces) ke liye `encodeURIComponent`:

```
js
document.cookie = encodeURIComponent("my name") + '=' + encodeURIComponent("Joh
```

Limitations:

- Ek baar me sirf **ek** cookie set/update
- `name=value` (encode ke baad) **4KB** se zyaada nahi
- Ek domain par kul cookies **~20+** (browser par depend)

Attributes `key=value` ke baad, `;` se:

```
js
```

```
document.cookie = "user=John; path=/; expires=Tue, 19 Jan 2038 03:14:07 GMT";
```

`domain`

`domain=site.com`: cookie kahan accessible hai.

- **Kisi dusre 2nd-level domain** (`other.com`) ko cookie kabhi nahi milegi (safety restriction).
- By default sirf usi domain par jisne set kiya, **subdomain** (`forum.site.com`) par bhi **nahi**.
- Subdomains ko dene ke liye root domain explicitly do:

```
js
```

```
document.cookie = "user=John; domain=site.com"; // ab *.site.com sab dekhenge
```

(Purani syntax `domain=.site.com` ab dot ignore hota hai.)

`path`

`path=/mypath`: sirf us path ke neeche ke pages par cookie dikhegi (absolute hona chahiye, default current path). `path=/admin` ho to `/admin` aur `/admin/something` par dikhega, `/home` ya `/` par nahi. Aam taur par `path=` rakho taaki sab pages par dikhe.

`expires`, `max-age`

Inme se koi na ho to cookie browser/tab band hote hi gayab (**session cookie**).

- `expires=Tue, 19 Jan 2038 03:14:07 GMT`: exact format, GMT me. `date.toUTCString()` se milta hai.

```
js
```

```
let date = new Date(Date.now() + 86400e3); // +1 din
document.cookie = "user=John; expires=" + date.toUTCString();
```

Past ki date se cookie delete.

- `max-age=3600`: abhi se kitne **seconds** baad. `0` ya negative se delete. Dono ho to `max-age` ki chalti hai.

secure

Cookie sirf **HTTPS** par jaayegi. Default me (`http://site.com`) par set cookie (`https://site.com`) par bhi dikhti hai (cookies protocol nahi dekhte). (**secure**) se HTTPS-set cookie HTTP par nahi jaayegi.

```
js
```

```
document.cookie = "user=John; secure";
```

samesite

XSRF (Cross-Site Request Forgery) se bachaav.

XSRF attack kya hai?

Aap (`bank.com`) par logged in ho (auth cookie hai). Dusri window me galti se (`evil.com`) par gaye, jisme JS ek form (`bank.com/pay`) par submit kar deta hai (hacker ke account me paisa). Browser (`bank.com`) ko har request me cookie bhejta hai chahe form (`evil.com`) se aaya ho, isliye bank aapko pehchan ke payment kar deta hai.

Asli banks me forms me special **XSRF token** hota hai. Par sab jagah lagana time leta hai.

samesite se

- **samesite=strict**: agar user site ke **bahar se** aaya (email ka link, (`evil.com`) ka form...) to cookie **kabhi nahi** bhejti. XSRF poori tarah fail. Chhoti takleef: legitimate link se aane par bhi site aapko nahi pehchanti. Workaround: do cookies, ek "general recognition" ke liye aur ek data badalne wale kaamo ke liye (**strict**).
- **samesite=lax** (bina value ke (**samesite**) bhi yahi): thoda relaxed, XSRF se bhi bachata hai aur user experience nahi bigadta. Cookie tab bhejta hai jab dono sach hon:
 1. HTTP method **safe** ho (jaise GET, POST nahi)
 2. **Top-level navigation** ho (browser address bar ka URL badle). (`<iframe>`) me ya JS network requests navigation nahi hote.

Yaani link se aane par cookie milti hai, par dusri site se network request ya form submission par nahi.

Drawback: bahut purane browsers (2017 ke aaspaas) (**samesite**) ignore karte hain. Isliye akele iske bharose mat raho, XSRF tokens ke saath extra layer ki tarah use karo.

httpOnly

JS se koi lena dena nahi, par completeness ke liye. Server (**Set-Cookie**) me set karta hai. **JS se cookie ka koi access nahi** (`document.cookie`) me dikhti nahi). Ye hacker ki injected JS se authentication cookie churane se bachata hai.

Appendix: Cookie helper functions

```
js

function getCookie(name) {
  let matches = document.cookie.match(new RegExp(
    "(?:^|; )" + name.replace(/[. $? *| {} \(\) \[ \] \\ \\/ + ^]/g, '\\\\$1') + "=([^;])"
  ));
  return matches ? decodeURIComponent(matches[1]) : undefined;
}

function setCookie(name, value, attributes = {}) {
  attributes = { path: '/', ...attributes };

  if (attributes.expires instanceof Date) {
    attributes.expires = attributes.expires.toUTCString();
  }

  let updatedCookie = encodeURIComponent(name) + "=" + encodeURIComponent(value);

  for (let attributeKey in attributes) {
    updatedCookie += "; " + attributeKey;
    let attributeValue = attributes[attributeKey];
    if (attributeValue !== true) updatedCookie += "=" + attributeValue;
  }

  document.cookie = updatedCookie;
}

setCookie('user', 'John', {secure: true, 'max-age': 3600});

function deleteCookie(name) {
  setCookie(name, "", { 'max-age': -1 });
}
```

Update ya delete karte waqt wahi `path` aur `domain` do jo set karte waqt the.

Appendix: Third-party cookies

Aisa cookie jo us domain ne set kiya ho jo **page ke domain se alag** hai.

1. `site.com` ke page par ``.
2. `ads.com` server `Set-Cookie: id=1234` deta hai (sirf `ads.com` par dikhega).
3. Agli baar `ads.com` par cookie bhejta hai, user pehchana gaya.

4. User `other.com` (jisme bhi `ads.com` ka banner hai) par jaye to `ads.com` phir usi cookie se pehchan leta hai, yaani **sites ke beech tracking**.

Isliye ads/tracking me use hote hain. Browsers me user disable kar sakta hai. **Safari** third-party cookies bilkul allow nahi karta, **Firefox** ki ek "black list" hai.

Dhyan: kisi third-party domain ki script (jaise `google-analytics.com/analytics.js`) agar `document.cookie` se cookie set kare to wo third-party nahi hai, wo **current page ke domain** ki hai.

Appendix: GDPR

JS se nahi, par dhyan rakhne layak. Europe ke GDPR ke hisaab se **tracking/identifying/authorizing cookies** ke liye user ki **explicit permission** chahiye. Sirf koi jaankari save karne wale (jo track ya identify nahi karte) cookies ke liye nahi. Do tarike:

1. Sirf authenticated users ke liye: registration form me "privacy policy accept" checkbox.
2. Sabke liye: newcomers ko modal splash screen dikhake sahmati lo.

Summary

- `document.cookie` se access. Write sirf mentioned cookie badalta hai.
- Name/value encode karo. Ek cookie max 4KB, ek domain par ~20+.
- Attributes: `path=`, `domain=site.com`, `expires`, `max-age`, `secure`, `samesite`.
- `httpOnly` server se, JS access band.
- Third-party cookies browsers block kar sakte hain. Tracking cookie ke liye EU me GDPR permission.